ScreenPilot: architecture

Frontier BOG/26 · Challenge 1 · Python, Playwright, FastAPI · Principle: think once, replay many (T1 scripted → T2 learned routes → T3 LLM explorer)

status + summary → GET /tasks/{id}

screenshots / mouse and keys over noVNC

Evaluator / user

Task in natural language +
desktop URL and password

Entrypoint API · FastAPI

Authorization: Bearer team token

`GET /health → 200`

`POST /tasks → {task_id}`
queued, returns in <100 ms (limit 2 s)

`GET /tasks/{id}`
en_curso | terminada | fallida
+ summary (README format)

Task queue

asyncio queue → worker pool (N)

1 · Session

Playwright (Python) opens noVNC, logs in, gets the canvas, takes screenshots, sends mouse and keys

2 · App fingerprint

Window title + OCR of the menu bar
→ app_id (known or new app)
→ loads that app's routes

3 · Router

Embeddings shortlist the top-3 routes
→ 1 cheap LLM call: pick a route and fill its slots (or "no match")

route found

no match

T2 · Route replayer (0–1 LLM calls)

Runs a stored chain of primitives with the slots filled, e.g. create_client(name, tax_id):

```
open_menu("Clientes > Nuevo")
fill_form({Nombre:$name, NIT:$id})
click_text("Guardar"); expect("ok")
```

Anchored on text + screen fingerprint, not x,y

replay breaks →
T3 resumes there
+ repairs route

T3 · Explorer loop (LLM, max 45 steps)

a. Perceive
screenshot → OCR, boxes

b. Set-of-marks
numbered elements + text

c. Planner LLM
picks the next primitive, e.g. click #14

d. Act + check
run the primitive, wait for a stable frame, compare

Cheap vision model by default → escalates to a stronger model after 2 failed steps. Loop detection: same screen and action 3 times → try a different strategy.

T1 · Primitives library (scripted, 0 tokens, works on any app)

`click_text · double_click · type · key · fill_form · open_menu · select_option · dismiss_popup`
`find_row · scroll_until · paginate · export_report · save_dialog · wait_stable · expect_text`
Every action uses OCR anchors and checks its own result. Deletes run only when the task asks for them.

4 · Verifier

Checks the end state on screen: success toast, new row, field value
→ terminada or fallida

5 · Route learner

Turns a successful T3 trace into a route: values → $slots, adds text anchors and expected screens

6 · Summary writer

Writes the README-format summary: what was done, data, evidence
→ updates the task record

save / repair route

Guardrails: time and cost budget per task

0–15 s: connect, fingerprint, route · replay target < 90 s · explorer at most 45 steps · hard stop at 4:30 → fallida + partial summary
Cost cap per task (US$0.40 explore / US$0.05 replay) · total budget US$45 · each action waits for a stable frame
Unexpected modal → dismiss_popup or read it and decide · same action 3 times → change strategy
Incident readiness (15:30): no fixed coordinates, routes keyed by text anchors, re-learning falls back to T3

WORKER / ORCHESTRATOR (one task per worker, 5-minute deadline)

Dev eval harness

`/__reset → POST /tasks`
`→ poll → /__check`

5 examples + ~15 paraphrases + new values, injected pop-ups, resized window. Logs success, time, cost and failure type.

Runs on every deploy.

Remote desktop (noVNC)

Legacy GUI app, no API, no docs
Test app on Railway; the evaluation app is unseen and reached the same way

`/__reset /__check`
(desktop token, used only by the harness)

Pixels only → screenshots + input events

OpenRouter (LLMs)

Cheap vision model: planner + router
Stronger model: only when stuck
Embedding model: shortlisting routes

US$45 budget (US$5 reserved for the hidden evaluation)

SQLite · persistent volume

`tasks`	id, status, summary, timings
`apps`	fingerprint, screen map
`routes`	steps, $slots, anchors, embedding, success rate
`runs`	route, steps, tokens, cost, time, failure type

Docker (Chromium) on Railway

Usage tracker

Ranks routes by use and success rate
→ hardens the top routes (more checks, fewer steps); demotes failing ones

→ failure types feed the dev loop
(fix the biggest category first)